EXPERIENCES.FUN

Spatial Explore Product & Engineering Specification

Turn the Explore globe from a decorative visualization into the spatial interface for Experiences.fun's real-world contribution graph.

■	↓	↓	↓
EXPERIENCE	**PARTICIPANTS**	**CONTRIBUTION**	**REPUTATION**

Core decision

Do not build a prettier globe. Build a spatial discovery engine where geography is the interface through which users discover experiences, communities, people, activity and measurable contribution.

Status: Recommended product direction

Audience: Product / engineering agent implementing the Explore experience

Platform constraint: Telegram Mini App first, with mobile WebView performance treated as a first-class requirement.

1. Executive direction

The current globe is an interactive SVG city visualization. It is useful, but it compresses too much of the underlying product into a handful of city markers. The next version should expose the actual network of Experiences.fun without turning the UI into a dashboard full of controls.

> **Product thesis**
>
> **Experiences.fun should make real-world contribution spatially visible.** The globe is the visual surface for that system, not the system itself.

The globe should progressively reveal detail as the user zooms:

- **Zoomed out:** cities, clusters and network activity.
- **Medium zoom:** categories, Nodes, experience density and active communities.
- **Close zoom:** individual experiences, people and contribution signals.
- **Experience level:** host, Node, participants, date, XP and action.

The key product transition is:

Globe as visualization	Globe as discovery engine
City markers	Experience + Node + people graph
Selection	Geographic navigation + contextual discovery
Static aggregates	Live activity + contribution signals

2. Current state and why it needs to change

The existing implementation is in **artifacts/mobile/components/ExploreGlobe.tsx**. It uses React Native + react-native-svg, hand-authored land shapes, a custom latitude/longitude projection, city markers, friend pins, graticules, zoom controls and a selected-city preview.

Its existing data model is city-centric. A city contains counts such as experiences, participants, places and XP offered. That means the globe cannot naturally expose the individual experiences that create those aggregates.

This creates three product limitations:

- **Loss of granularity:** several experiences become one city dot.
- **Weak exploration loop:** the user sees where activity exists but not why they should tap it.
- **Disconnected social graph:** friends are pins instead of participants in the same experience/community network.

> **Do not remove the current renderer immediately**
>
> Keep the SVG globe as a fallback and low-performance mode. The WebGL version should be introduced behind a shared GlobeEngine interface so the product can degrade gracefully.

Existing strengths to preserve:

- Approximate city locations for privacy.
- City selection and bottom-sheet preview.
- Friend pins and profile interaction.
- Reset / zoom controls.
- Responsive fullscreen behavior.

3. Product model

Model the Explore experience as a graph rather than a list of geographic points.

```
World
  ■■■ Country
       ■■■ City
            ■■■ Community / Node
                 ■■■ Experience
                      ■■■ Host
                      ■■■ Participants
                      ■■■ Contributions
                      ■■■ XP
                      ■■■ Reputation
```

Recommended spatial entities:

Experience	A real-world activity users can join	Marker / cluster
Node	Verified community place or organization	Persistent place marker
User / Friend	Social participation and reputation	Avatar / social pin
Activity	Recent contribution or participation	Pulse / ring
Mission	Cross-location contribution objective	Mission layer
City	Aggregation and navigation boundary	Cluster / label

The globe should answer four questions quickly:

- What is happening near this place?
- Who is participating?
- What can I contribute to?
- What impact or XP can I earn?

4. Globe modes

Avoid a control panel with ten filters. Use four high-level modes that change what the globe emphasizes.

Explore	What can I do here?	Experiences + Nodes + clusters
Social	Where are my people?	Friends + communities + social arcs
Activity	What is happening now?	Live pulses + recent participation
Impact	Where is contribution happening?	XP + missions + contribution density

> **Default state**
>
> **Explore** should be the default. It is the shortest path from opening the page to finding a real-world activity.

Mode switching should not replace search or category chips. Categories remain useful as lightweight filters inside Explore.

5. Interaction specification

Drag	Rotate globe smoothly	Spatial browsing
Pinch / wheel	Zoom with bounded camera distance	Reveal more detail
Tap cluster	Zoom toward cluster or open city sheet	Progressive disclosure
Tap city	Fly camera to city	City becomes spatial context
Tap experience	Open experience sheet	View details + join
Tap Node	Open Node/community sheet	Discover local ecosystem
Tap friend	Open profile / recent activity	Social discovery
Search place	Geocode/resolve known city and fly there	Search becomes navigation
Reset	Return to global camera	Recover browsing context

Progressive disclosure rule:

```
global → city cluster → category cluster → experience → Node / people → action
```

The globe should never force the user to understand the entire network before taking a simple action.

6. Technical architecture

```
Explore
■■■ GlobeEngine
■   ■■■ WebGLRenderer
■   ■■■ SVGFallback
■■■ SpatialData
■   ■■■ Experiences
■   ■■■ Nodes
■   ■■■ Users
■   ■■■ Friends
■   ■■■ Cities
■   ■■■ Missions
■   ■■■ Activity
■■■ SpatialLayers
■   ■■■ ExperienceLayer
■   ■■■ NodeLayer
■   ■■■ FriendLayer
■   ■■■ ActivityLayer
■   ■■■ HeatmapLayer
■   ■■■ MissionLayer
■   ■■■ JourneyLayer
■■■ GlobeModes
■   ■■■ Explore
■   ■■■ Social
■   ■■■ Activity
■   ■■■ Impact
■■■ Search
■■■ CityView
■■■ ExperienceSheet
```

Recommended initial renderer: **react-globe.gl** / ThreeJS / WebGL. It provides points, arcs, polygons, paths, heatmaps, hex bins, labels, HTML elements, 3D objects and custom layers, making it a strong MVP foundation for this product direction. ■cite■turn0search1■turn0search4■

Do not hard-code the renderer into the product model. Create a GlobeEngine interface so a future custom Three.js / React Three Fiber renderer can replace it without rewriting the spatial data or product logic.

Recommended component boundary:

```
  mode={mode}
  experiences={experiences}
  nodes={nodes}
  friends={friends}
  activity={activity}
  selectedLocation={selectedLocation}
  performanceTier={performanceTier}
  onExperiencePress={...}
  onNodePress={...}
  onFriendPress={...}
  onLocationChange={...}
/>
```

7. Data model changes

Introduce an experience-level spatial model and derive city aggregates from it instead of treating the city aggregate as the source of truth.

```
type GlobeExperience = {
  id: number;
  title: string;
  latitude: number;
  longitude: number;
  city: string;
  country: string;
  category: ExperienceCategory;
  xpReward: number;
  participantCount: number;
  availablePlaces: number;
  startDate: string;
  hostId: number;
  nodeId?: number;
  status: ExperienceStatus;
};
```

Derived views:

```
GlobeExperience[]
  ■■■ cluster(experiences)
  ■■■ aggregateByCity(experiences)
  ■■■ aggregateByCategory(experiences)
  ■■■ heatmap(experiences)
  ■■■ activityStream(experiences + participation events)
```

Privacy:

- Do not expose exact home locations for users.
- Friends should use approximate or experience-associated locations unless the user explicitly shares more precise location.
- Continue the existing approximate-city-location baseline for privacy-sensitive data.

Node examples:

- Schools
- Libraries
- Community centres
- Universities
- Charities / nonprofits
- Businesses
- Parks
- Creative studios
- Web3/community hubs

8. Visual design direction

The visual goal is not geographic realism. It is a recognizable **digital social planet** that makes human activity feel alive.

Recommended visual language:

Background	Deep navy / near-black space
Globe	Dark geographic surface with subtle atmospheric rim
Primary activity	Experiences.fun pink #F20789
Secondary activity	Purple #6805F2
Network / info	Aqua #04ADBF
Activity	Soft rings, pulses and trails
Labels	Sparse, contextual, never cluttered
Graticules	Subtle or removed

Avoid:

- Google Earth imitation.
- Large persistent grids.
- Too many floating labels.
- Permanent particle effects that add no meaning.
- Heavy 3D assets before the discovery loop works.

9. Features that make the globe strategically valuable

These features should be implemented as layers over the same spatial model. The goal is to compound the value of the globe rather than build separate mini-products.

Node layer

- Make verified real-world places first-class entities.
- Example city view: **London Network** → 42 Nodes · 184 Experiences · 2,481 Participants · 38,420 XP · 12 active today.
- Node tap opens a community sheet with experiences, people and contribution history.

Activity layer

- Show recent activity as short-lived pulses or rings.
- Examples: someone joined an experience, completed a mission, earned XP, checked into a Node, or published a contribution.

Mission layer

- Turn geography into a game board for contribution objectives.
- Examples: complete three environmental experiences across London, visit two community Nodes, or contribute to a local education mission.

Journey / reputation layer

- Visualize a user's contribution history as a path across places and experiences.
- Keep blockchain mechanics invisible unless they matter. Show **+250 XP**, not transaction hashes.

10. Search + city experience

The existing Explore search should become a geographic command bar. Search is not only text filtering. It is a navigation mechanism.

```
Search: "London"
        ↓
Resolve place
        ↓
Fly globe to London
        ↓
Show city context
        ↓
Experiences + Nodes + People + Activity
        ↓
User chooses an action
```

City sheet should include:

- City name + country
- Experience count
- Participants / sign-ups
- Node count
- XP currently offered
- Active experiences
- Top categories
- Friends / people active nearby
- Primary CTA: Explore experiences

The city sheet is a contextual layer. The globe remains the primary navigation surface.

NON-NEGOTIABLE

11. Telegram Mini App performance requirements

Telegram's current Mini App guidance emphasizes responsive mobile-first design, smooth animations ideally around 60fps, safe-area handling and adapting effects based on Android performance class. Telegram exposes LOW, AVERAGE and HIGH performance classes for Android Mini Apps. ■cite■turn0search0■

Therefore the globe must have explicit performance tiers:

HIGH	WebGL + atmosphere + subtle particles + live pulses + richer transitions
AVERAGE	WebGL + reduced particles + capped markers + simplified effects
LOW	SVG fallback or extremely simplified WebGL + minimal animation

Implementation rules:

- Use clustering to avoid rendering hundreds of DOM/HTML elements simultaneously.
- Throttle camera and pointer-driven state updates.
- Avoid React re-rendering the entire globe on every drag frame.
- Lazy-load heavy globe dependencies where practical.
- Pause expensive animations when the Mini App is inactive.
- Respect Telegram safe areas and fullscreen behavior.
- Test on real iOS and Android devices, not only desktop Chrome.

12. Implementation roadmap

P0	GlobeEngine + WebGL renderer + SVG fallback	Globe works across performance tiers
P0	Experience-level spatial data + clustering	Individual experiences can appear without marker overload
P0	Fly-to-city + improved selection	City selection changes camera and context
P0	Node layer	Community places are discoverable
P1	Explore / Social / Activity / Impact modes	Modes change visible data without new screens
P1	Activity heatmap + live pulses	Recent activity is visually obvious
P1	Search → geographic navigation	Searching a place flies globe and updates context
P1	City ecosystem sheet	City becomes a discovery hub
P2	Friends/social graph	People are connected to experiences and Nodes
P2	Missions + contribution arcs	Globe supports game loops
P2	Journey / reputation layer	Contribution history is spatially visible
P3	3D buildings / richer assets / advanced effects	Only added after core loop proves valuable

Do not start with P3

3D buildings, satellite imagery and elaborate visual effects are presentation multipliers. They do not solve the core discovery problem. Prove the spatial contribution loop first.

13. Agent acceptance criteria

1. The Explore page can render a WebGL globe on supported devices and automatically fall back to SVG when required.

2. Experiences are represented at experience level, with clustering at lower zoom levels.

3. City aggregates are derived from underlying experiences rather than being the only spatial data source.

4. Tapping a city flies the camera to that city and opens contextual city information.

5. Tapping an experience opens the existing experience detail flow.

6. Nodes are represented as first-class geographic entities.

7. Friends can be rendered as a social layer without exposing precise private locations.

8. Search can resolve a city/place and navigate the globe to it.

9. Explore, Social, Activity and Impact modes can switch the visible globe layers.

10. The globe remains usable with hundreds of experiences through clustering and progressive disclosure.

11. Android LOW / AVERAGE / HIGH performance classes receive appropriate visual complexity.

12. Telegram safe areas and fullscreen behavior are respected.

13. The globe does not block the main Explore feed or bottom navigation.

14. The implementation keeps the renderer replaceable through a GlobeEngine abstraction.

Suggested QA scenarios:

- Cold launch in Telegram on low-end Android.
- Open Explore with 500+ experiences.
- Rotate globe continuously for 10–15 seconds.
- Rapid pinch zoom in/out.
- Switch modes while globe is moving.
- Search for a city and immediately tap an experience.
- Enter fullscreen and rotate device.
- Switch Telegram light/dark theme.
- Open and close the Mini App repeatedly.

DECISION

14. Final recommendation

> **Build this as a spatial social operating layer, not a globe feature.**
>
> The durable architecture is: **spatial data** → **layers** → **modes** → **geographic navigation** → **contextual sheets** → **contribution actions**. The renderer is replaceable; the product graph is not.

Recommended architecture summary:

```
Experience data
      ↓
Spatial indexing / clustering
      ↓
GlobeEngine
  ■■■ WebGL
  ■■■ SVG fallback
      ↓
Modes
  ■■■ Explore
  ■■■ Social
  ■■■ Activity
  ■■■ Impact
      ↓
Context
  ■■■ City
  ■■■ Node
  ■■■ Experience
  ■■■ Person
      ↓
Action
  ■■■ Join
  ■■■ Contribute
  ■■■ Follow
  ■■■ Earn XP
```

External implementation references:

Telegram Mini Apps documentation: https://core.telegram.org/bots/webapps

react-globe.gl documentation / package: https://www.npmjs.com/package/react-globe.gl

react-globe.gl GitHub: https://github.com/vasturiano/react-globe.gl

Source notes: Telegram documentation confirms mobile-first responsiveness, smooth animations, safe-area handling, fullscreen support and Android performance-class optimization. react-globe.gl currently provides a React + ThreeJS/WebGL globe with points, arcs, polygons, heatmaps, labels, HTML elements and custom layers. ■cite■turn0search0■turn0search1■turn0search4■

END OF SPECIFICATION